Hashfuncs

Fast fingerprints, buckets & cache keys.

Install & load

```
SET allow_extension_repositories = 'allowed';
CREATE EXTENSION REPOSITORY IF NOT EXISTS query_farm WITH PREFIX 'https://duckdb-extensions.query.farm';
INSTALL hashfuncs FROM query_farm;  LOAD hashfuncs FROM query_farm;
```

01 At a glance

Hash SQL values with named non-cryptographic algorithms. Choose a stable representation and seed for repeatable partitioning, probabilistic-filter inputs and compact fingerprints.

Canonical input → Named hash → Bucket / key

02 Core SQL surface

`xxh3_64(value [, seed])`

Fast 64-bit result as UBIGINT.

`xxh3_128(value [, seed])`

128-bit result as UHUGEINT.

`xxh3_128_hex(text [, seed])`

Canonical 32-character lowercase hex digest for strings and BLOBs.

`xxh32 / xxh64`

Classic xxHash variants, with optional seed overloads.

`rapidhash(value [, seed])`

64-bit RapidHash for general-purpose fingerprints.

`murmurhash3_32 / murmurhash3_x64_128`

Named MurmurHash3 variants for explicit algorithm selection.

`spark_hash / iceberg_bucket / kafka_partition`

Reproduce Spark, Iceberg and Kafka hashing; farm_fingerprint matches BigQuery.

`jump_hash(key, buckets)`

Consistent hashing: adding a shard moves about 1/(n+1) of keys.

03 Keep in mind

Keep the representation fixed

Match input bytes, type and seed: 0.0 and -0.0, or NaNs with different bits, hash differently, so normalize when SQL-equal values must match. Hex output is canonical high64 then low64; numeric UHUGEINT output keeps its legacy layout for stored keys.

04 Route to 16 buckets

```
SELECT i, xxh3_64(i, 42) % 16 AS bucket
FROM range(8) AS t(i);
```

The same input type and seed assign each value to the same bucket.

05 Format a string fingerprint

```
SELECT xxh3_128_hex('hello') AS digest;
-- 32 lowercase hexadecimal characters
```

Matches canonical xxHash digests from Python, Rust and C.

06 Choose the output width

```
SELECT typeof(xxh3_64('hello')) AS small,
       typeof(xxh3_128('hello')) AS large;
-- UBIGINT, UHUGEINT
```

Choose the width for your collision budget and storage format.

07 Match another system

```
SELECT spark_hash('Spark', [123], 2) AS h;
-- -1321691492, as in Spark's hash()
```

For Spark bucketing use ((h % n) + n) % n; iceberg_bucket(value, n) follows the Iceberg spec.

08 In practice

Collisions are possible

Hashes are compact fingerprints, not uniqueness constraints. Retain original keys for exact equality. Use crypto for authentication or adversarial inputs.

Full docs & examples ↗ Source ↗